

YUANJUN (JOHNNY) DAI

216-269-2394 • yxd429@case.edu • Cleveland, OH
github.com/Johnny-dai-git • johnny-dai-git.github.io/portfolio • [LinkedIn](#)

SUMMARY

AI Infrastructure and Applied AI Engineer, PhD candidate at CWRU. Specializes in the security, optimization, and observability of ML/LLM training and inference infrastructure. 8 research papers (7 first-author) at IEEE, ACM, and Springer; **2+ years of systems software engineering** at Cisco and Broadcom; 2+ years of **cross-functional** product ownership at Midea and Xiaomi.

EDUCATION

- Ph.D. Candidate in Computer Science (AI Infrastructure)

Case Western Reserve University

08/2019 – Present
- M.S. in Electrical and Computer Engineering (Computer Architecture)

University of Pittsburgh
- B.S. in Electrical Engineering

Northeastern University

SKILLS

Languages	Python, C/C++
ML Frameworks	PyTorch (DDP/FSDP), vLLM, DeepSpeed, Megatron-LM, NCCL, BytePS, scikit-learn
Parallelism Paradigms	Data Parallel (DDP/FSDP), Model Parallel (TP/PP), Pipeline Parallel (DeepSpeed), Hybrid Parallel
Infrastructure	Kubernetes, Docker, CI/CD Pipelines, ArgoCD, Slurm, HPC
Systems & APIs	FastAPI, REST API, gRPC
Observability	eBPF (Kernel Tracing), Prometheus, Grafana, DCGM Exporter, Intel RAPL
Cloud & HPC	Lambda Labs HPC, Ohio Supercomputer Center, NSF FABRIC, CloudLab
Networking	SDN (Software-Defined Networking), P4, NPU/ASIC (Broadcom BCM SDK)

RESEARCH & SYSTEMS ENGINEERING EXPERIENCE

- Research Assistant, AI Infrastructure & ML Systems

Case Western Reserve University
- DYNAMIX – RL-Based Adaptive Batch Scheduling for Distributed ML | [Paper](#) / [Code](#)

Designed a PPO-based centralized RL scheduler monitoring real-time hardware signals (CPU/memory usage, network bandwidth, retransmission rate via custom eBPF probes) and statistical efficiency signals (loss convergence rate, generalization performance) to dynamically adapt batch size; co-scales learning rate proportionally (Linear Scaling Rule) to maintain training stability; evaluated on CNN architectures across CIFAR-100 and similar benchmarks using both Ring AllReduce (PyTorch DDP) and Parameter Server (BytePS) across **8-64 A100 GPUs**; achieves **46% end-to-end training time reduction** and improved final model accuracy over fixed batch size baseline.

PRISM – Priority-Based Selective Reliability for Distributed ML Training | [Paper](#) (Under Review)

Designed PRISM, a transport-layer library (NCCL/Gloo → UDP) hooking into PyTorch DDP's AllReduce at bucket-ready events to provide priority-based selective reliability without modifying training code; classifies gradients by magnitude into high-priority (full SACK-based retransmission) and low-priority (local compensation via cached values); supports kernel-bypass (LibVMA) and kernel-based (GSO/GRO) variants; achieves **8-15% throughput gain for CNNs and 15-21% for LLMs** over TCP under packet loss, matching NDP latency at 144-worker scale on commodity hardware.

Training-Time Side-Channel Attacks on Distributed ML Systems | [Paper](#) / [Code](#)

Developed deep understanding of DML computation and communication behavior in MXNet BSP PS to expose MLaaS training-phase side-channel vulnerability; (1) applied **K-means clustering** on push/pull network traffic to identify silent periods (forward pass computation without communication); (2) applied **Fourier transform** on Intel RAPL CPU and memory power signals for noise reduction, extracting layer-by-layer energy consumption patterns; (3) trained a **Decision Tree classifier** on energy features across Conv and FC layers under varying parameter sizes for layer-type classification; (4) built **polynomial regression models** on CPU energy profiles of activation functions for activation function classification; reconstructed complete target DNN architecture; achieves **>99% layer-type accuracy and <1.5% tensor-size error** across ZFNet, AlexNet, and VGG.

Priority-Based eBPF Network Flow Telemetry for AI / DML Infrastructure | [Paper](#) / [Code](#)

Designed and implemented PSketch, the first in-kernel priority-aware sketch data structure using eBPF; high-priority flows tracked precisely via BPF_HASH priority table (O(1) lookup), low-priority flows estimated via 3-layer Count-Min Sketch; voting-based hash collision resolution and atomic operations for thread-safe in-kernel updates; achieves **96% Top-K accuracy, 96.4% retransmission recall**, and <1.1% throughput overhead at 10 Gbps on FABRIC testbed with CAIDA 2019 backbone traffic.

High-Resolution eBPF System Resource Profiling for DML Workloads | [Paper](#) / [Code](#)

Designed eHashPipe, a dual-pipeline eBPF observability framework for DML workloads — (1) a HashPipe-based memory profiler tracking Top-K memory-intensive processes via cascading slot eviction with ptr2size deallocation tracking, and (2) a per-thread

CPU tracker hooking sched_switch tracepoint for thread-level CPU time attribution aggregated per PID; atomic operations for thread-safe in-kernel updates; delivers **10–14× finer temporal resolution** than top/htop, ~11ms latency, and >90% Top-K fidelity on commodity Linux servers.

- **Scotch – Elastic SDN Overlay for Control-Plane Scalability** | [Paper](#) / [Code](#)

Designed a distributed control-plane overlay using OvS-based traffic offloading and large-flow migration; **reduced controller congestion by >70%** on a 34-node GENI Clos topology.

- **Enterprise-Grade LLM Production Deployment Platform on Kubernetes** | [Code](#)

Built a K8s microservices platform (Gateway / Router / Worker) supporting vLLM and TensorRT-LLM inference with ArgoCD GitOps, Prometheus/Grafana observability, and CI/CD automation across **50+ Kubernetes deployments**.

- **Domain-Adapted LLMs for Clinical Risk Adjustment Coding** | [Paper](#)

Fine-tuned PubMedBERT and Qwen-7B for HCC coding from EHR notes; achieved **macro-/micro-F1 of 0.775/0.742** and **0.81/0.802** on MIMIC-IV with imbalance-aware training and per-label calibration.

- **BiLSTM-Based Side-Channel Modeling for Distributed ML Analysis** | [Paper](#)

Designed a BiLSTM pipeline aligning CPU/memory energy traces with per-layer timestamps; reliably distinguishes Conv, FC, and activation layers during distributed ML training using ensemble learning.

INDUSTRY EXPERIENCE

Technical Product Manager

07/2018 – 01/2019

Xiaomi

- Market-facing PM role: conducted competitive analysis and user research to define product requirements for a smart IoT appliance.
- Assessed technical feasibility and complexity of proposed features; delivered initial technical proposals covering architecture trade-offs and implementation risks to align engineering and business stakeholders.

Product Manager

09/2016 – 06/2018

Midea Group

- Served as technical PM for **Fun Oven II**, a multi-disciplinary smart appliance spanning data collection, ML model training & deployment, mobile app integration, cloud backend, and culinary algorithm design.
- Participated in resolving technical challenges across all dimensions — from training AI image recognition models and deploying Docker services on Alibaba Cloud, to defining cooking curves and app UX.
- Product debuted at **2018 AWE Show** and won the **Red Dot Design Award** · [Red Dot ↗](#) · [PR Newswire ↗](#)

Software Engineer II

06/2015 – 08/2016

Cisco Systems, Inc.

- Developed NPU/ASIC features for a 100 Gbps core router (Azure, AWS, Alibaba Cloud) using Broadcom BCM SDK.
- Contributed to QoS and ECMP modules, reducing forwarding latency under heavy load.

Software Engineer Intern

10/2014 – 05/2015

Broadcom (Emulex)

- Replaced polling-based data acquisition with a DMA + circular-buffer architecture in an RTOS pipeline, **reducing CPU load by 30%** and improving sampling rate and control-loop responsiveness.
- Ported a Linux-only management CLI for BladeEngine ASIC-based CNA hardware to Windows Server, adapting system calls, threading models, and error handling.

Engineer Intern

01/2014 – 04/2014

Tenova I2S

- Re-architected an RTOS data acquisition pipeline using DMA and circular buffers, **reducing CPU load by 42%** and improving sampling rate and control-loop responsiveness.

ADDITIONAL SYSTEMS ENGINEERING EXPERIENCE

16-bit MIPS-Like Pipelined CPU Design and Implementation · [Report ↗](#)

- Designed an end-to-end pipelined CPU from custom ISA through RTL, synthesis, and layout; 5-stage pipeline with hazard handling and branch prediction, achieving **~3× speedup** over single-cycle design.

PUBLICATIONS

8 papers total, 7 first-author, 5 published at IEEE CCNC, IEEE ICNC, ACM TOIT, and Springer SecureComm; 3 under review (incl. ACM TOPS). Full list: johnny-dai-git.github.io/portfolio